

Project 2 Supplementary Material

Instructor: Reza Faieghi

Inputs and Outputs

As the first step to implement the flight simulator physics engine, we should know what the inputs and outputs of the physics engine software are.

The inputs include the following variables, all of which will be commanded from a joystick:

- $\delta_{T,1}$: Engine 1 throttle angle
- $\delta_{T,2}$: Engine 2 throttle angle
- δ_A : Aileron deflection angle
- δ_E : Elevator deflection angle
- δ_R : Rudder deflection angle

Remark 1: Note that all these inputs are angles, and the appropriate unit to use them in the simulator is radians. However, for convenience, in this document and further communications, we will use angles and the software must convert angles into radians.

Remark 2: You can still include the above inputs in your simulator before connecting the joystick. For this purpose, you can use various blocks available in the “source” category of blocks in Simulink. For example, you can set constant values for each input, and change these blocks with joystick readings at the time of integrating a joystick with your software.

With the physics engine, our goal is to take the above five inputs and predict the 12 variables of aircraft that describe six degrees-of-freedom flight. These variables are therefore the outputs of the physics engine that will be sent to FlightGear. These variables are calculated from the following equations:

- The *force equation* to calculate translational velocity components:
 - U : longitudinal speed,
 - V : lateral speed,
 - W : vertical speed.
- The *moment equation* Rotational velocity otherwise known as body rate components:
 - P : roll rate,
 - Q : pitch rate, and
 - R : yaw rate.
- The *kinematic equation* Attitude angles otherwise known as Euler angles:
 - ϕ : roll angle,
 - θ : pitch angle, and
 - ψ : yaw angle.
- The *navigation equation* to calculate the position of aircraft in NED coordinates frame on Earth

- p_N : the “North” component of the position vector.
- p_E : the “East” component of the position vector, and
- h : altitude, which is the “Down” component of the position vector but in negative direction.

Note that the NED position coordinates will be later translated into geodetic coordinates so that our simulator can work with GPS coordinates.

Remark 3: Before connecting your software to Flight Gear, you can still visualize your software outputs by using scope blocks.

Constants and basic variables

Before implementing the main equations, we need to define several constants and basic variables that will be used in the flight equations.

The aircraft under consideration is a narrow-body twinjet airliner, with the following properties:

- Mass: $m = 120000 \text{ kg}$,
- Inertia matrix: $J = m \begin{pmatrix} 40.07 & 0 & -2.0923 \\ 0 & 64 & 0 \\ -2.0923 & 0 & 99.92 \end{pmatrix} \text{ kgm}^2$,
- Mean aerodynamic chord: $\bar{c} = 6.6 \text{ m}$ ($\bar{c}$ will be used in the definition of several other parameters below),
- Coordinates of aircraft center of mass: $cm = (0.23\bar{c}, 0, 0.1\bar{c})^T$,
- Coordinates of aircraft aerodynamic center: $ac = (0.12\bar{c}, 0, 0)^T$,
- Coordinates of engine 1 thrust application point: $apt_1 = (0, -7.94, -1.9)^T$,
- Coordinates of engine 2 thrust application point: $apt_2 = (0, 7.94, -1.9)^T$,
- Wing planform area: $S = 260 \text{ m}^2$,
- Wing planform area of tail: $S_t = 64 \text{ m}^2$,
- Generalized length of wings: $l = 6.6 \text{ m}$,
- Distance between the aircraft cm and the ac of tail: $l_t = 24.8 \text{ m}$,
- Angle of attack when lift is zero: $\alpha_0 = -11.5^\circ$,

- Air density is $\rho = 1.225 \text{ kg/m}^3$, and
- $g = 9.81 \text{ m/s}^2$.

Remark 4: Note that the angle of attack must be converted into radians. Always pay careful attention to the units of the parameters. Remember that the Mars Climate Orbiter mission failed because a variable was not presented in an appropriate unit within the system software.

In addition to the above constants, there are several basic variables that need to be calculated during flight simulation. These variables will be used in several equations, especially in calculating the aerodynamic coefficients. Here are these variables listed with their formulae:

- Airspeed: $V_T = \sqrt{U^2 + V^2 + W^2}$ (Can you guess where this equation is coming from?),
- Dynamic pressure: $\bar{q} = \frac{1}{2} \rho V_T^2$,
- Angle of attack: $\alpha = \tan^{-1}\left(\frac{W}{U}\right)$, and
- Sideslip angle: $\beta = \sin^{-1}\left(\frac{V}{V_T}\right)$.

At the point where we will calculate the aerodynamic coefficients, we need to model the effects of the aircraft tail separately. As such, there are two additional variables that need to be calculated to be used in computing the aerodynamic coefficients. They include:

- Downwash angle of the wings: $\varepsilon = 0.25(\alpha - \alpha_0)$, and
- Angle of attack of the tail: $\alpha_t = \alpha - \varepsilon + \delta_E + 1.3Q \frac{l_t}{V_T}$.

With these constants and basic variables implemented, we can now calculate the aerodynamic coefficients.

Aerodynamic coefficients

- Lift coefficient is calculated as the sum of the lift coefficient of the wing body and the tail as $C_L = C_{L,wb} + C_{L,t}$. These coefficients are given as follows

$$C_{L,wb} = 5.5(\alpha - \alpha_0), \text{ and}$$

$$C_{L,t} = 3.1 \frac{S_t}{S} \alpha_t,$$

- Drag coefficient:

$$C_D = 0.13 + 0.07(5.5\alpha + 0.654)^2,$$

- Crosswind coefficient:

$$C_C = -1.6\beta + 0.24\delta_R,$$

- Rolling moment coefficient:

$$C_l = -1.4\beta - 11 \frac{l}{V_T} P + 5 \frac{l}{V_T} R - 0.6\delta_A + 0.22\delta_R,$$

- Pitching moment coefficient:

$$C_m = -0.59 - 3.1 \frac{S l_t}{S l} (\alpha - \varepsilon) - 4.03 \frac{S l_t^2}{S l^2} \frac{l}{V_T} Q - 3.1 \frac{S l_t}{S l} \delta_E, \text{ and}$$

- Yawing moment coefficient:

$$C_n = (1 - 3.8179\alpha)\beta + 1.7 \frac{l}{V_T} P - 11.5 \frac{l}{V_T} R - 0.63\delta_R.$$

We are now ready to calculate aerodynamic forces and moments.

Aerodynamic forces

For the aircraft under consideration, the aerodynamic forces are modeled via wind tunnel testing. Therefore, once we calculate the drag (D), crosswind (C), and lift (L) forces, an additional step is required to express these forces in FRD coordinates where the force equations are defined. This becomes clear in the following expressions.

First, calculate drag, crosswind, and lift forces in the wind coordinate as follows:

$$D = \bar{q} S C_D, C = \bar{q} S C_C, \text{ and } L = \bar{q} S C_L.$$

Then, use the following expressions to resolve the D , C , and L in FRD coordinates. The resulting forces will be the real drag (X_A), crosswind (Y_A), and lift (Z_A) forces during the flight, and will be forces that needs to be fed into the force equations.

$$X_A = -D \cos \alpha \cos \beta + C \cos \alpha \sin \beta + L \sin \alpha,$$

$$Y_A = -D \sin \beta - C \cos \beta, \text{ and}$$

$$Z_A = -D \sin \alpha \cos \beta + C \sin \alpha \sin \beta - L \cos \alpha.$$

Note that the index A indicates the aerodynamic source of the forces. We will use the index T to indicate the engine thrust source of similar terms in the following pages.

Aerodynamic moments

Let us denote the rolling, pitching, and yawing moments with l , m , and n .

Remark 5: The above symbols are commonly used to denote the aerodynamic moments; however, note that m is also used to denote the mass of aircraft and you should use different variable names in your software to differentiate between mass and moment.

The aerodynamic moments will be calculated in multiple steps. First, use the aerodynamic coefficients and aircraft constants to calculate the following moments:

$$l'_A = \bar{q} \bar{c} SC_l, m'_A = \bar{q} \bar{c} SC_m, \text{ and } n'_A = \bar{q} \bar{c} SC_n.$$

Now, calculate the actual aerodynamic moments using the following vector operation:

$$\begin{pmatrix} l_A \\ m_A \\ n_A \end{pmatrix} = \begin{pmatrix} l'_A \\ m'_A \\ n'_A \end{pmatrix} + \begin{pmatrix} X_A \\ Y_A \\ Z_A \end{pmatrix} \times \begin{pmatrix} 0.11\bar{c} \\ 0 \\ 0.1\bar{c} \end{pmatrix}$$

Keep in mind that $\times$ is the cross product between the two vectors. The reason that we added the cross-product term is to account for the moments generated due to the center of mass of the aircraft. Note the center of mass of the aircraft is not necessarily the aerodynamic center of the aircraft. In fact, the right most term in the above equation is the distance between these two points obtained from the constant values defined earlier in this document. We assume that the aerodynamic forces act on the center of mass of the aircraft. At the same time, we assume that the aerodynamic moments act on the aerodynamic center of the aircraft. Therefore, from the aerodynamic center point of view, there will be moments generated by the aerodynamic forces applied to the center of mass of the aircraft, and the cross-product term in the above equation accounts for that.

Engine forces

The thrust generated by each engine is aligned with the X axis. Therefore, Y_T and Z_T will be zero. X_T can be found as follows.

$$X_T = X_{T,1} + X_{T,2} = \delta_{T,1}mg + \delta_{T,2}mg,$$

where $\delta_{T,1}$ and $\delta_{T,2}$ are the throttle settings of each engine 1 and 2, respectively. The acceptable range of throttle settings is $0.5 \leq \delta_{T,i} \leq 1$ which can be modeled by using a saturation block in Simulink. (For your information, for $\delta_{T,i} = 1$, the thrust-over-weight ratio is about 0.35). Further, the engine cannot immediately respond to throttle settings, rather there is a limited response rate. This can be modeled by using a rate limiter block in Simulink. Set the maximum rising slew rate to 1.6 and falling slew rate to -1.6.

Engine moments

For each engine, there will be a moment generated as follows.

$$\begin{pmatrix} l_{T,i} \\ m_{T,i} \\ n_{T,i} \end{pmatrix} = \begin{pmatrix} X_{T,i} \\ 0 \\ 0 \end{pmatrix} \times (cm - apt_i)$$

The total rolling, pitching, and yawing moments generated by engines will be summation of the corresponding moments of the two engines.

Weight

The effects of the weight of the aircraft are incorporated in the force equations directly as can be seen in the force equations.

Force equations

The force equations are given as follows:

$$\begin{aligned}\dot{U} &= RV - QW - g \sin \theta + \frac{X_T + X_A}{m}, \\ \dot{V} &= -RU + PW + g \sin \phi \cos \theta + \frac{Y_T + Y_A}{m}, \\ \dot{W} &= QU - PV + g \cos \phi \cos \theta + \frac{Z_T + Z_A}{m}.\end{aligned}\tag{1}$$

Note that force equations model the effects of total forces acting on the aircraft center of mass. In class, we talked about the different sources of forces during flight. They include gravity, engine thrust, and aerodynamics, all of which can be found in the above equations. The terms with g are related to weight. The terms with indices T and A relate to engine thrust, and aerodynamic forces resolved along X , Y , and Z axes of the FRD coordinate system.

Moment equations

Assume that the inertia matrix of the aircraft is given as follows:

$$J = \begin{pmatrix} J_x & J_{xy} & J_{xz} \\ J_{xy} & J_y & J_{yz} \\ J_{xz} & J_{yz} & J_z \end{pmatrix}.\tag{2}$$

It is worth mentioning that the inertia matrix is always symmetric.

Let us define an auxiliary variable as $\Gamma = J_x J_z - J_{xz}^2$. Then, the moment equations of aircraft are given as follows:

$$\begin{aligned}\Gamma \dot{P} &= J_{xz} [J_x - J_y + J_z] PQ - [J_z (J_z - J_y) + J_{xz}^2] QR + J_z (l_A + l_T) + J_{xz} (n_A + n_T), \\ J_y \dot{Q} &= (J_z - J_x) PR - J_{xz} (P^2 - R^2) + m_A + m_T, \\ \Gamma \dot{R} &= [(J_x - J_y) J_x + J_{xz}^2] PQ - J_{xz} [J_x - J_y + J_z] QR + J_{xz} (l_A + l_T) + J_x (n_A + n_T).\end{aligned}\tag{3}$$

It is interesting to note that the moments equations model the effect of moment during flight, and they only include aircraft body rates, moments, and aircraft inertia matrix. The physical interpretation of the inertia matrix is that it models the distribution of the mass of aircraft in X , Y , and Z directions.

Kinematic equations

The kinematic equations do not have any force or moment terms and are given as follows:

$$\begin{aligned}\dot{\phi} &= P + \tan \theta (Q \sin \phi + R \cos \phi) \\ \dot{\theta} &= Q \cos \phi - R \sin \phi \\ \dot{\psi} &= \frac{Q \sin \phi + R \cos \phi}{\cos \theta}\end{aligned}\quad (4)$$

Navigation equations

The navigation equations are given as follows:

$$\begin{aligned}\dot{p}_N &= U \cos \theta \cos \psi + V(-\cos \phi \sin \psi + \sin \phi \sin \theta \cos \psi) + W(\sin \phi \sin \psi + \cos \phi \sin \theta \cos \psi), \\ \dot{p}_E &= U \cos \theta \sin \psi + V(\cos \phi \cos \psi + \sin \phi \sin \theta \sin \psi) + W(-\sin \phi \cos \psi + \cos \phi \sin \theta \sin \psi), \\ \dot{h} &= U \sin \theta - V \sin \phi \cos \theta - W \cos \phi \cos \theta.\end{aligned}\quad (5)$$

Note that the navigation equations are obtained by multiplying a 3D rotation matrix to U , V , and W . You can try this on your own as an exercise.

Geodetic coordinates equations

Knowing the positions in the imaginary NED coordinates frame that we imposed on Earth surface is not much helpful. By converting these coordinates to geodetic coordinates, we can use GPS to allow our flight simulator work directly with GPS coordinates.

To convert NED position coordinates to GPS coordinates, we will need to use a spheroid model of Earth. Spheroid is an oval that is rotated along of its axes to create a 3D shape just like how sphere is created from a circle.

The GPS uses an Earth model that is documented in a standard called WGS 84: Department of Defence World Geodetic System 1984 (available online). According to this document, the Earth equatorial radius a is measured at

$$a = 6378137 \text{ m.} \quad (6)$$

To define longitude and latitude, there is an additional parameter required from the Earth's shape which is called the first eccentricity denoted by e . According to WGS 84, the first eccentricity for Earth is

$$e = 0.081819190842622. \quad (7)$$

Note that e is a dimensionless number and does not have a unit.

Suppose that an aircraft is at a position that has a longitude λ . Using a set of geometric operations, for the aircraft location, the following parameters can be computed.

The meridian radius at that location given by

$$M = \frac{a(1-e^2)}{\sqrt{(1-e^2 \sin^2 \phi)^3}}. \quad (8)$$

The prime vertical radius at that location given by

$$N = \frac{a}{\sqrt{1 - e^2 \sin^2 \phi}}. \quad (9)$$

By calculating these parameters, we can implement equations that will convert the NED coordinates into GPS coordinates. Let us denote latitude with μ , then we have:

$$\begin{aligned} \dot{\mu} &= \frac{\dot{P}_N}{M + h} \\ \dot{\lambda} &= \frac{\dot{P}_E}{(N + h) \cos \mu} \end{aligned} \quad (10)$$

Note that the altitude h remains the same in both NED and GPS coordinates.

Remark 6: If some parts of the above equations are not clear, especially with regards to the geodetic coordinates, please treat them as a black-box and implement them. More explanations will be provided in the upcoming classes.

Remark 7: As you are new to MATLAB and Simulink, it is likely that you do not know how to implement certain parts of the equations during the software development. This is completely fine, and it is part of the learning process. Try finding solutions via internet search, using MATLAB help, brainstorming with your team members, and consulting with your TA and instructor.

Validating the Software

To ensure your software implementation is correct, use the following initial conditions and inputs, and simulate for 50 seconds. Your plots should match the trajectories given in Figure 2.

Remark 8: It is important to note that the given inputs must remain constant for during simulation. Therefore, to validate the software, you do not need to use a joystick.

```
longitude0 = deg2rad(-79.625335);
latitude0 = deg2rad(43.676856);
altitude0 = 7224;
phi0 = 0;
theta0 = 0.1;
psi0 = 0;
U0 = 84.9905;
V0 = 0;
W0 = 1.2713;
P0 = 0;
Q0 = 0.0149;
R0 = 0;
deltaT10 = 0.0821;
deltaT20 = 0.0821;
deltaA0 = 0;
deltaE0 = -0.1780;
deltaR0 = 0;
```

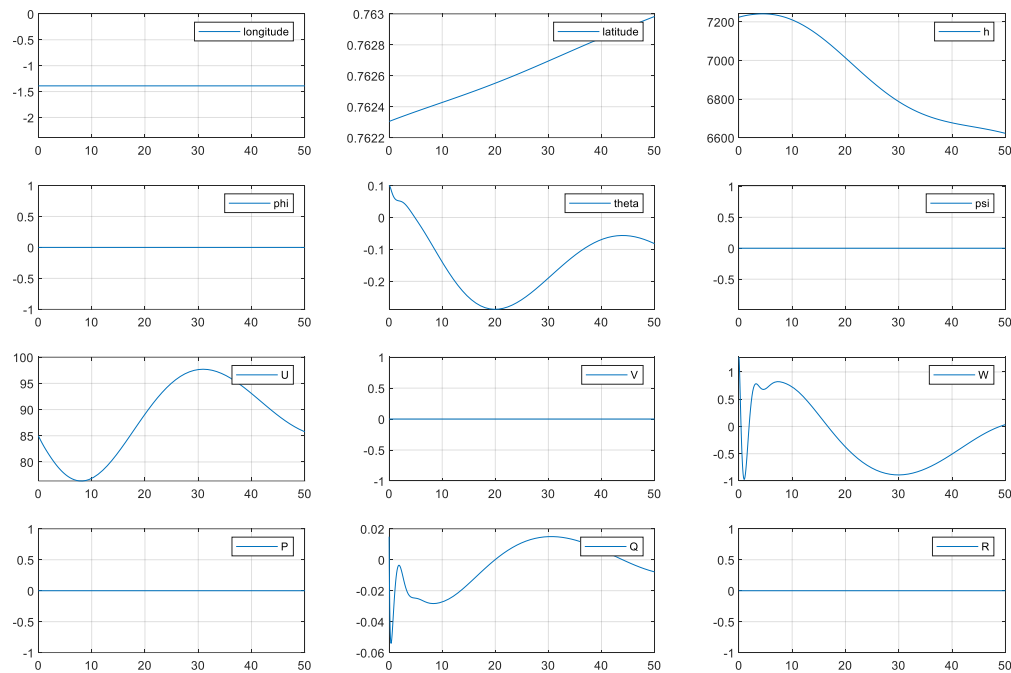



Figure 2: Time response of the different states of the aircraft for the given initial conditions and constant inputs

Connecting the Software to FlightGear

There are multiple steps required to connect your Simulink software to FlightGear.

On the Simulink side of things, you need to use two blocks:

- FlightGear Preconfigured 6DoF Animation, and
- Generate Run Script

Remark 9: If you have a Simulation Pace block in your Simulink file already (likely to handle joystick inputs) you need to once you use the FlightGear block. The reason is that the FlightGear block itself has a Simulation Set Pace block and this will conflict with another Set Pace block.

It is important to feed the information in appropriate order to the FlightGear block. The order of the inputs can be found in the block help. It can be also inferred from the sample Simulink file provided.

For the Generate Run Script make sure you are using appropriate aircraft model. The default aircraft model in FlightGear is Cessna 172. If you like to use this aircraft model, then the aircraft name in the Generate Run Script block must be set to “c172p”.

Alternatively, you can use a Boeing 757-200 model for your simulator. For that, you need to download the aircraft model (the link to a Google Drive folder is provided below). Once you have downloaded the aircraft model, it is important to follow the following steps in order.

- Navigate to the FlightGear installation directory. The default installation directory is “C:\Program Files\FlightGear 2020.3”.
- Within the FlightGear directory, navigate to Data\Aircraft.
- Unzip the provided Boeing 757-200 model and copy the folder to the Aircraft folder.
- Launch FlightGear app.
- On the left-side menu of the FlightGear menu, choose Aircraft settings, and then select Boeing 757-200.
- Press the “Fly” button at the left corner of the FlightGear interface and wait until the scene is loaded. This will ensure that FlightGear has successfully loaded the aircraft model.

Once the aircraft model setup is complete, in Simulink, and in the Generate Run Script, change the aircraft name to “b757-200-RB211”.

Once the aircraft model is finalized, you can press the Generate Script button in the Generate Run Script block menu. This will create a file with extension .bat. By clicking on this file, FlightGear will run. For convenience, this script file is already generated and included in the Google Drive folder shared below.

The first time running FlightGear with the script, allow an ample amount of time for downloading scenery. To help with this process, a folder of FlightGear scenery is also provided in the Google Drive folder. Navigate to FlightGear installation directory, and under the Data folder find the Scenery folder. Replace this folder with the provided Scenery folder. Remember to first unzip the Scenery folder that is shared via Google Drive and then copy it into the Data folder of FlightGear installation directory.

After completing the above steps and once FlightGear scene is up (but is in the frozen state), you can start the simulation in Simulink, and the aircraft should be in motion within the FlightGear environment.

For convenience, the following files are shared:

- **AircraftModel.slx:** A Simulink file with appropriate input-output setup. This is my own implementation of the simulator, but the flight equations are deliberate removed so that you can add your own!
- **RunAircraftModel.m:** In my own implementation of the simulator, this m-file defines all the initial conditions and inputs, runs the aircraft model, gathers the data and plots them.
- **runfg - b757.bat:** This is the script which runs FlightGear connected to Simulink. This file is for the Boeing 757 model.
- **FlightGear-2020.3.12.exe:** Installation file for the version of FlightGear that I used. You can also download this from the FlightGear website. You can also try older versions. The older versions have lower computing requirements.
- **757-200.zip:** The Boeing757-200 model file.
- **Scenery.zip:** This is scenery files for Toronto.

Remark 10: When unzipping the above files, make sure you do not have cascaded folders of the same name. For example, when you click on 757-200 folder, there should not be another sole folder named 757-200.

The link to Google Drive folder:

<https://drive.google.com/drive/folders/1-SSIE5XUnK2uAijE3BNt5GTT4Tc5PWJH?usp=sharing>